Análisis de Componentes Principales (PCA)

Aprendizaje No Supervisado · Reducción de Dimensionalidad · Perfil Sonoro de Canciones

Inteligencia Artificial · Sexto Semestre · Ingeniería de Sistemas de Información · PUCE

Objetivos de Aprendizaje

Al finalizar esta clase serás capaz de:

1. Explicar qué es el aprendizaje no supervisado y por qué conviene reducir la dimensionalidad de un dataset.
2. Describir los pasos de PCA: estandarizar, encontrar direcciones de máxima varianza, proyectar y elegir k.
3. Aplicar PCA en un Pipeline e interpretar la varianza explicada, los loadings y el biplot.
4. Entrenar el modelo final con el 100% de los datos, exportarlo con joblib y usarlo con canciones nuevas.

Aprendizaje Supervisado vs. No Supervisado

Hasta ahora cada modelo tenía una variable objetivo y; hoy no hay y

Aprendizaje no supervisado: el algoritmo recibe solo X (sin etiqueta y) y descubre por sí mismo la estructura de los datos.

Aspecto	Supervisado (clases anteriores)	No supervisado (hoy)
¿Tiene y?	Sí: convocado, aprobado…	No
¿Qué aprende?	Una función f(X) → y	La estructura de X
¿Cómo se evalúa?	Accuracy, F1, RMSE	Varianza explicada, interpretación
Ejemplos	Regresión logística, Random Forest	PCA, K-Means, DBSCAN

Dos Familias de Tareas No Supervisadas

Agrupar observaciones o resumir variables

Clustering (agrupamiento)

- Forma grupos de observaciones parecidas.
- Algoritmo de ejemplo: K-Means.
- Uso: segmentar oyentes según sus gustos musicales.

Reducción de dimensionalidad

- Resume muchas variables en pocas variables nuevas.
- Algoritmo de hoy: PCA.
- Uso: ver las 10 variables de audio de una canción en un gráfico 2D.

El Problema: Demasiadas Variables

spotify_audio_features.csv: cada canción se describe con 10 números

1 Visualización imposible

Una persona puede ver 2 o 3 ejes a la vez: 10 variables no caben en un gráfico.

2 Variables redundantes

energy y loudness_db tienen correlación 0.95: casi dicen lo mismo dos veces.

3 Maldición de la dimensionalidad

Con más variables los datos quedan más dispersos y se necesitan muchas más filas para aprender patrones confiables.

Objetivo: menos variables, perdiendo la menor información posible

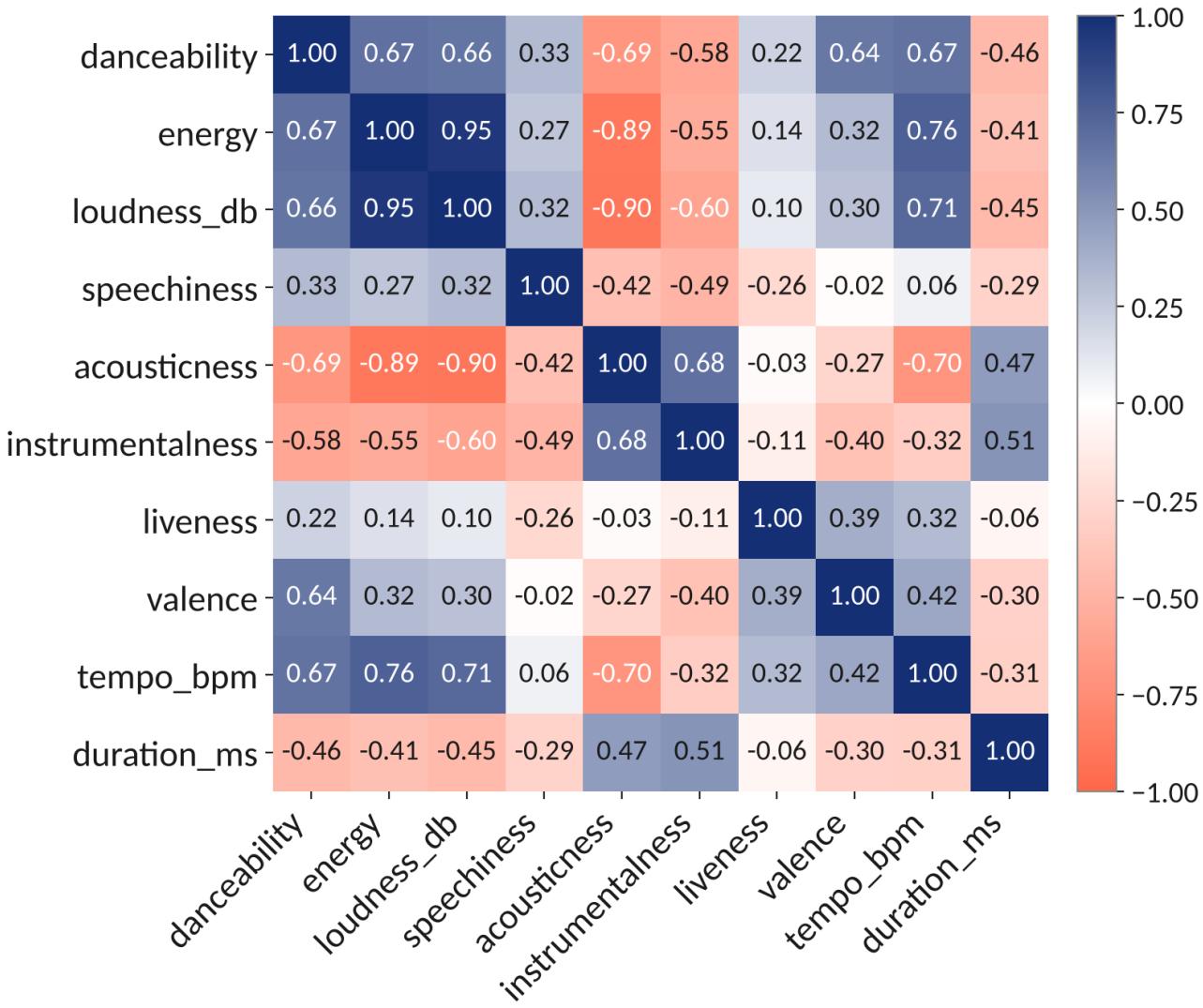
Dataset — Perfil Sonoro de 1 200 Canciones

spotify_audio_features.csv · 8 géneros (150 canciones c/u) · sintético · sin nulos

Variable	Rango en los datos	Qué mide
song_id	0 a 1 199	Identificador: no entra en PCA
genre	8 géneros (texto)	No entra en PCA: solo para colorear
danceability	0.07 a 1	Qué tan bailable es
energy	0 a 1	Intensidad percibida
loudness_db	−28.7 a −5.0 dB	Volumen promedio
speechiness	0 a 0.42	Presencia de palabras habladas
acousticness	0 a 1	Qué tan acústica es
instrumentalness	0 a 1	Probabilidad de no tener voz
liveness	0 a 0.41	Presencia de público en vivo
valence	0.07 a 1	Positividad (alegría) musical
tempo_bpm	80 a 170	Pulsaciones por minuto
duration_ms	121 265 a 238 049	Duración en milisegundos

Variables Redundantes: Matriz de Correlaciones

Correlación de Pearson real de las 10 variables · rango −1 a +1



Lectura

- energy y loudness_db: **+0.95**.
- loudness_db y acousticness: **−0.90**.
- energy y acousticness: **−0.89**.
- liveness es la menos relacionada: su máximo es 0.39 (con valence).
- Cerca de ±1 = redundancia**: PCA las combina en una sola componente.**

¿Qué es PCA?

PCA = Principal Component Analysis (Análisis de Componentes Principales)

Definición: técnica que transforma variables posiblemente correlacionadas en un conjunto menor de variables nuevas no correlacionadas, llamadas componentes principales (PC), ordenadas de mayor a menor varianza.

En lenguaje simple

- Cada componente es una mezcla ponderada de las 10 variables: en vez de 10 números por canción, usamos 2 o 3.

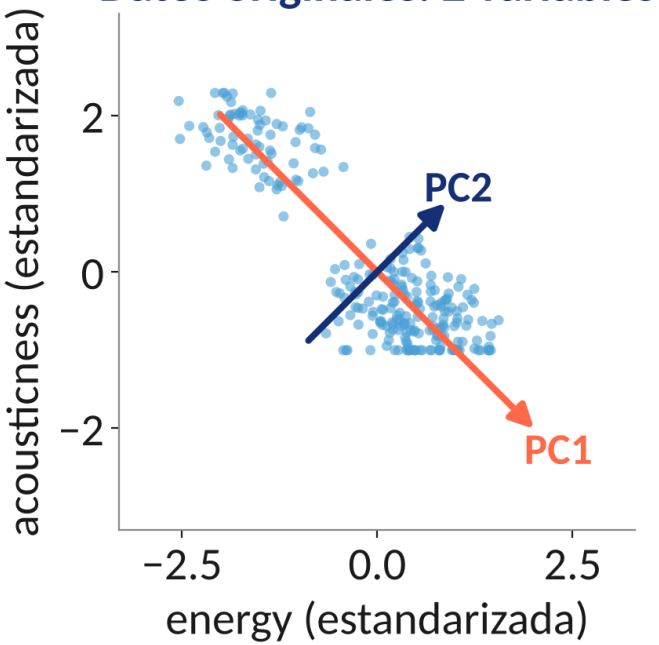
No correlacionadas	Ordenadas por varianza	Combinación lineal
Cada PC aporta información distinta.	PC1 captura más que PC2, y así sucesivamente.	Suma ponderada de las variables originales.

Fuente: Jolliffe, I. T. (2002). Principal Component Analysis (2.ª ed.). Springer.

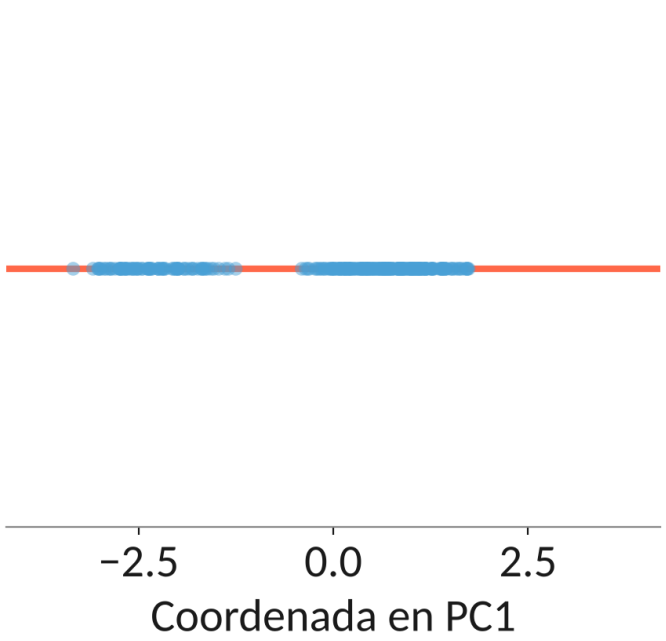
Intuición: Proyectar para Conservar Información

Ejemplo real con 2 variables del dataset (260 canciones al azar)

Datos originales: 2 variables



Proyección en PC1: 1 variable conserva el 94% de la varianza



Analogía: una foto

- Una foto pasa un edificio 3D a 2D.
- Un buen ángulo muestra más detalle.
- PCA elige el ángulo que conserva más varianza.

- energy y acousticness (r = −0.89) se resumen en un solo eje: PC1 conserva el 94% de su varianza.

PCA no elimina variables: crea variables nuevas que combinan las originales

Paso 1 — Estandarizar las Variables

Antes de PCA todas las variables deben tener la misma escala

Estandarización (StandardScaler)

$$z = \frac{x - \mu}{\sigma}$$

Lectura

- μ = media · σ = desviación estándar.
- Resultado: media 0 y desviación 1.

Sin estandarizar (prueba real)

- duration_ms va de 121 265 a 238 049; danceability de 0.07 a 1.
- PC1 explica ≈ 100% y es casi solo duration_ms.

Con estandarizar

- Todas las variables empiezan con el mismo peso.
- PC1 explica el 52.1% y combina casi todas las variables.

Regla: PCA siempre después de StandardScaler

Pasos 2 y 3 — Direcciones de Máxima Varianza

PCA busca los ejes donde las canciones se dispersan más

Cada componente es una combinación lineal de las variables estandarizadas (z)

$$\mathrm{PC}_1 = w_1\, z_1 + w_2\, z_2 + \cdots + w_p\, z_p$$

PC1	PC2
• Dirección de máxima varianza. • En los datos: 52.1% de la varianza.	• Perpendicular a PC1: no correlacionada. • Captura la mayor varianza restante: 16.0%.

Loadings (w): los pesos de cada variable en una componente; PCA los obtiene a partir de la matriz de correlaciones.

Pasos 4 y 5 — Proyectar y Elegir k

k = número de componentes que se conservan

Paso 4 — Proyectar

- Cada canción pasa de 10 números a k números.
- Ejemplo real (song_id 2, Reggaetón): PC1 = −2.11 y PC2 = −1.07.

Paso 5 — Elegir k

- Se conservan las k primeras PC.
- Regla práctica: acumular entre 80% y 90% de la varianza.
- En los datos: 4 PC = 84.8%.

Varianza explicada (λ = varianza de cada PC) · rango 0 a 1 · todas suman 1

$$\text{varianza explicada}_j = \frac{\lambda_j}{\lambda_1 + \lambda_2 + \cdots + \lambda_p}$$

El Flujo Completo de PCA

Cinco pasos, de 10 variables a k componentes



Entrada: matriz X de n filas × p variables (aquí 1 200 × 10).

Salida: matriz Z de n filas × k componentes no correlacionadas (aquí 1 200 × 2).

En scikit-learn: Pipeline con StandardScaler() y PCA(n_components=k), luego fit_transform(X)

La Clase PCA()

PCA no recibe y: fit() y fit_transform() solo usan X (las 10 variables numéricas)

```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

X_esc = StandardScaler().fit_transform(X)   # paso 1
pca = PCA(n_components=2)                   # 2 componentes
Z = pca.fit_transform(X_esc)                # pasos 2 a 4
```

Atributo: existe solo después de fit()	Qué devuelve
explained_variance_ratio_	Proporción de varianza de cada PC
components_	Loadings: una fila por PC, una columna por variable
n_components_	Número de PC conservadas

⚠ **Ojo:** explained_variance_ratio_ y components_ NO se pasan al crear PCA(): se leen después de fit().

Parámetros y Métodos de PCA()

Parámetros: se pasan al crear el objeto, antes de fit()

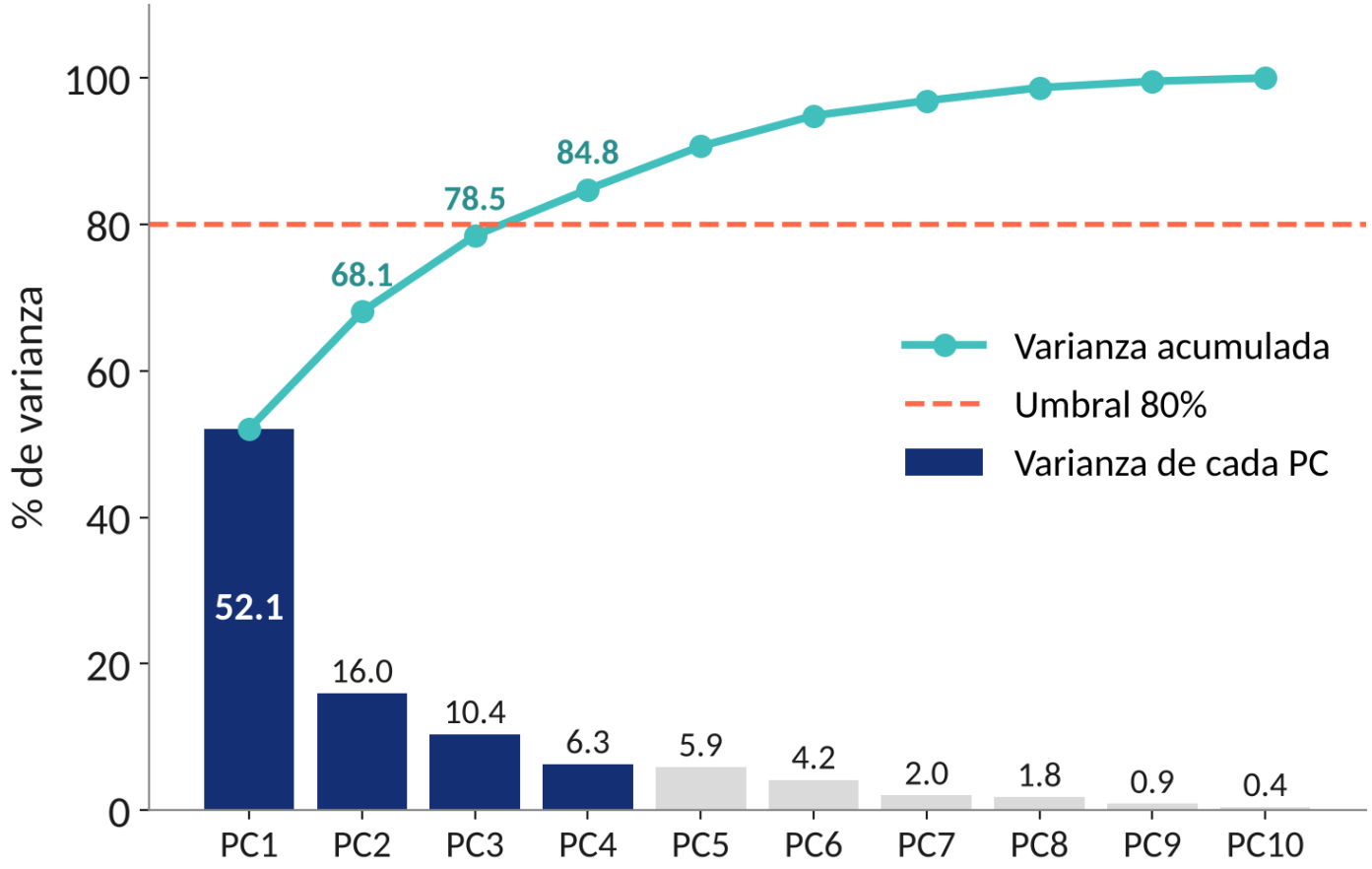
Parámetro	Defecto	Qué controla
`n_components`	None	None = todas | 2 = dos PC | 0.80 = las PC que sumen 80%
`whiten`	False	Lleva cada PC a varianza 1 (no se usa en el curso)
`random_state`	None	Semilla, solo para algunos métodos de cálculo

Método	Qué hace
`fit(X)`	Aprende las direcciones (componentes) a partir de X
`transform(X)`	Proyecta datos, incluso canciones nuevas, sobre esas direcciones
`fit_transform(X)`	Las dos cosas en un solo paso

Parámetro (se pasa al crear) ≠ atributo (existe después de fit())

Varianza Explicada: ¿Cuánta Información Conserva cada PC?

PCA con las 10 componentes sobre los datos estandarizados



PC	Acum.	¿Basta?
PC1	52.1%	No
PC1–PC2	68.1%	Mapa 2D
PC1–PC3	78.5%	Casi
PC1–PC4	84.8%	Sí (80%)
PC1–PC5	90.7%	Sí (90%)

Cada PC: 0% a 100% (0 a 1 en scikit-learn) · todas suman 100% · conservar 80%–90%

¿Cuántos Componentes Conservar?

Un decimal en n_components elige k solo · Pipeline: visto en clases anteriores

```
pca_80 = Pipeline([('escalar', StandardScaler()),
                   ('pca', PCA(n_components=0.80))])
pca_80.fit(X)
print(pca_80.named_steps['pca'].n_components_)   # 4
```

Para visualizar: k = 2

- 68.1% de la varianza.
- Suficiente para un mapa 2D.

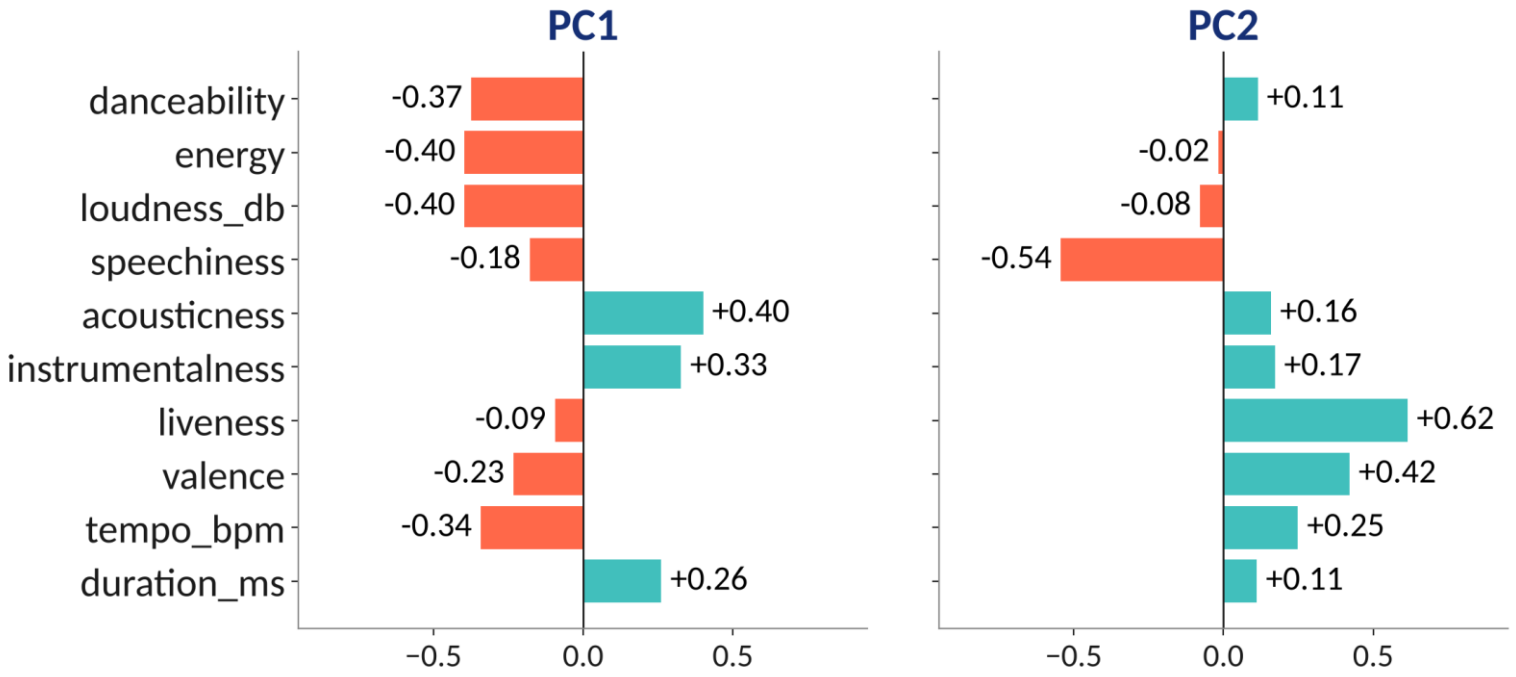
Para un modelo: k = 4

- 84.8% de la varianza.
- Reduce 10 variables a 4 antes de entrenar.

named_steps['pca']: recupera el paso PCA de un Pipeline ya entrenado.

Loadings: ¿Qué Hay Dentro de cada Componente?

pca.components_ · rango −1 a +1 · valores reales



⚠ **Signo arbitrario:** otra versión u otro programa puede invertir los signos de una PC (con scikit-learn < 1.5, PC2 sale invertida); la interpretación no cambia.

Lectura

Cómo leer un loading:

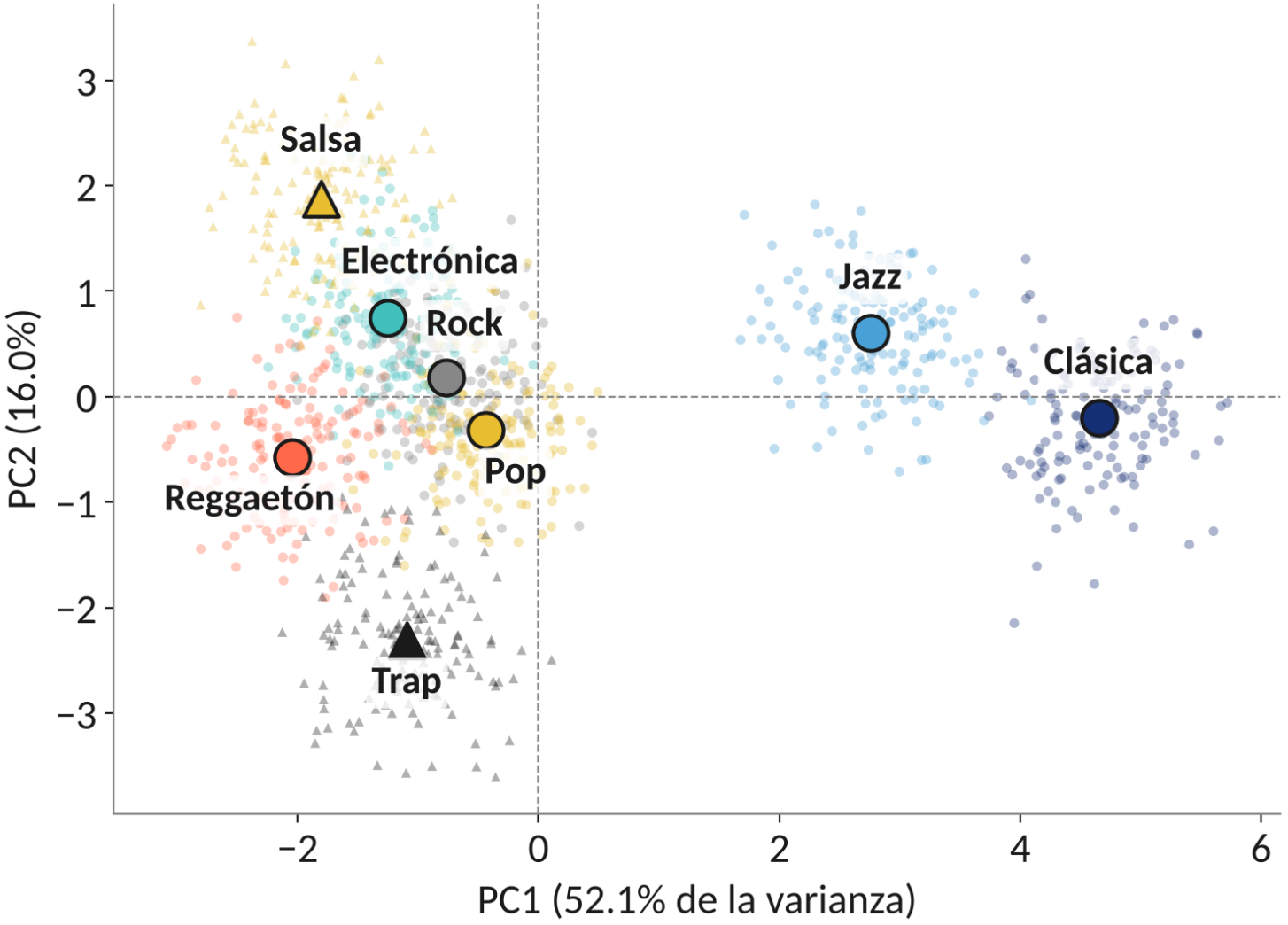
- **Signo** = hacia qué lado empuja la variable.
- **Tamaño** (sin importar el signo) = cuánto influye.
- **Cerca de 0** = casi no influye. Ej.: liveness en PC1 (−0.09).

Qué significa cada componente:

- **PC1:** canciones acústicas (+) frente a energéticas (−).
  - acousticness +0.40 · energy −0.40 · loudness_db −0.40
- **PC2:** canciones en vivo y alegres (+) frente a habladas (−).
  - liveness +0.62 · valence +0.42 · speechiness −0.54

Mapa de Canciones: PC1 vs. PC2

1 200 canciones en 2 coordenadas · centroide = promedio del género

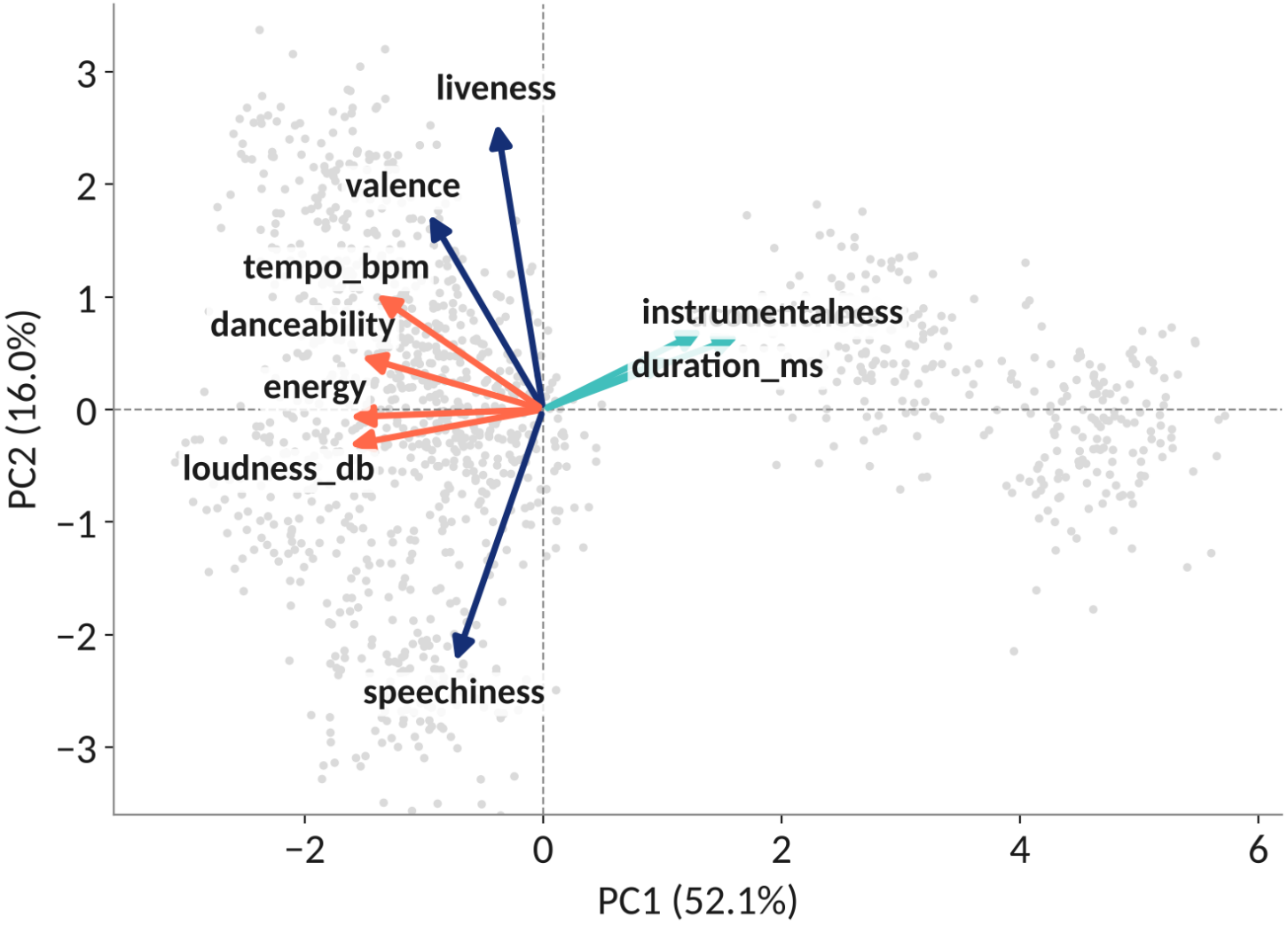


Lectura

- **PC1:** Clásica (4.65) y Jazz (2.75) en el lado acústico.
- **PC1:** Reggaetón (−2.05) y Salsa (−1.81) en el lado energético.
- **PC2:** Trap abajo (−2.31), mucha voz hablada.
- **PC2:** Salsa arriba (1.88), alegre y en vivo.
- El género NO se usó en PCA: solo colorea.

El Biplot: Canciones y Variables a la Vez

Puntos = canciones · flechas = loadings de cada variable



Cómo leerlo

- **Flecha larga** = variable influyente.
- **Misma dirección** = correlación positiva.
- **Opuestas** = correlación negativa.
- **Perpendiculares** = sin relación.
  - energy y loudness_db casi se superponen (r = 0.95).
  - Es aproximado: el plano solo muestra el 68.1% de la varianza.

Flujo Completo (1/3): Datos y X

Construir X solo con las 10 variables numéricas

```
import pandas as pd

df = pd.read_csv('spotify_audio_features.csv')
X = df.drop(columns=['song_id', 'genre'])   # 10 numéricas
generos = df['genre']                       # solo para colorear el mapa
```

- song_id es un identificador y genre es texto: ninguno entra en PCA.
- No hay train/test: no existe una y que predecir; se usan las 1 200 canciones.

> ⚠ **Si PCA va antes de un modelo supervisado:** el Pipeline se ajusta solo con X_train, como cualquier preprocesamiento.

Flujo Completo (2/3): Pipeline Escalar + PCA

StandardScaler y PCA() en un solo objeto

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

pca_pipe = Pipeline([('escalar', StandardScaler()),
                     ('pca', PCA(n_components=2))])
Z = pca_pipe.fit_transform(X)          # 1 200 filas x 2

mapa = pd.DataFrame(Z, columns=['PC1', 'PC2'])
mapa['genero'] = generos               # para colorear
```

- fit_transform(): aprende medias, desviaciones y direcciones, y transforma X.
- mapa: tabla de 1 200 filas con PC1, PC2 y el género de cada canción.

Flujo Completo (3/3): Varianza, Loadings y Centroides

named_steps['pca'] recupera el paso PCA ya entrenado

```
pca = pca_pipe.named_steps['pca']
print(pca.explained_variance_ratio_)    # ≈ [0.521 0.160]

loadings = pd.DataFrame(pca.components_.T,
                        index=X.columns,
                        columns=['PC1', 'PC2'])
print(loadings.round(2))

centroides = mapa.groupby('genero').mean()
print(centroides.round(2))
```

- .T (transpuesta): una fila por variable y una columna por PC.
- Los resultados son los de las diapositivas de loadings y del mapa por género.

¿Cuándo Usar PCA y Cuándo No?

PCA trabaja con variables numéricas y relaciones lineales

✓ Usar PCA cuando…

- Hay muchas variables numéricas correlacionadas.
- Se quiere visualizar datos en 2D o 3D.
- Se busca reducir ruido o costo antes de un modelo.
- Hay multicolinealidad en una regresión.

✕ No usar PCA cuando…

- Las variables son texto o categorías.
- Hay que explicar el modelo con las variables originales.
- Las relaciones son no lineales: usar t-SNE o UMAP.
- Hay valores nulos sin imputar.

t-SNE y UMAP: técnicas de reducción no lineal (t-distributed Stochastic Neighbor Embedding; Uniform Manifold Approximation and Projection).

PCA en Aplicaciones Reales

Una técnica de 1901 que sigue en uso

Reconocimiento facial (Eigenfaces): cada rostro se resume en pocas componentes; 96% de acierto con cambios de iluminación.

Genómica: PC1 y PC2 de ~197 000 variantes genéticas de 1 387 europeos reproducen el mapa de Europa.

Finanzas: tres componentes (nivel, pendiente y curvatura) resumen casi todo el movimiento de las tasas de interés de los bonos.

Origen: Karl Pearson propuso la idea en 1901 como «líneas y planos de mejor ajuste».

Fuentes: Turk y Pentland (1991), J. Cognitive Neuroscience · Novembre et al. (2008), Nature · Litterman y Scheinkman (1991), J. Fixed Income · Pearson (1901), Philosophical Magazine.

La Clase NearestCentroid(): el Género más Cercano

La usaremos al final para ubicar canciones nuevas en el mapa

NearestCentroid: calcula el centroide (promedio) de cada género y asigna cada canción al género cuyo centroide está más cerca.

```
from sklearn.neighbors import NearestCentroid

nc = NearestCentroid()          # valores por defecto
nc.fit(Z, generos)              # centroides en PC1-PC2
print(nc.predict(Z[:3]))        # género más cercano
```

⚠ **Ojo:** PCA no usa el género; NearestCentroid sí lo usa para calcular los centroides (atributo centroids_). Aquí es una referencia de cercanía en el mapa, no un clasificador evaluado.

Entrenamiento Final con el 100% de los Datos

El modelo final se entrena con las 1 200 canciones y se exporta

Análisis (exploración)	Modelo final (100%)
pca_pipe: sirvió para entender la varianza y los loadings.	modelo_final: escalar + PCA + género cercano.

```
from sklearn.neighbors import NearestCentroid

modelo_final = Pipeline([('escalar', StandardScaler()),
                         ('pca', PCA(n_components=2)),
                         ('genero', NearestCentroid())])
modelo_final.fit(X, generos)   # 100%: las 1 200 canciones
```

Observa: en PCA no hubo división train/test, así que el análisis ya usó el 100%. El modelo final repite ese ajuste y agrega el paso del género cercano para usarlo con datos nuevos.

Exportación con joblib (1/2)

Un solo archivo .pkl guarda el escalador, el PCA y los centroides

```
import joblib
joblib.dump(modelo_final, 'modelo_pca_spotify.pkl')

# en otro programa (ej. un servicio web):
import joblib
import pandas as pd
modelo = joblib.load('modelo_pca_spotify.pkl')
```

Qué se guarda	Qué NO hace falta
El Pipeline completo, ya entrenado.	Estandarizar a mano: el Pipeline lo hace.

⚠ **Versión:** carga el .pkl con la misma versión de scikit-learn con la que se guardó.

Uso con Canciones Nuevas (2/2)

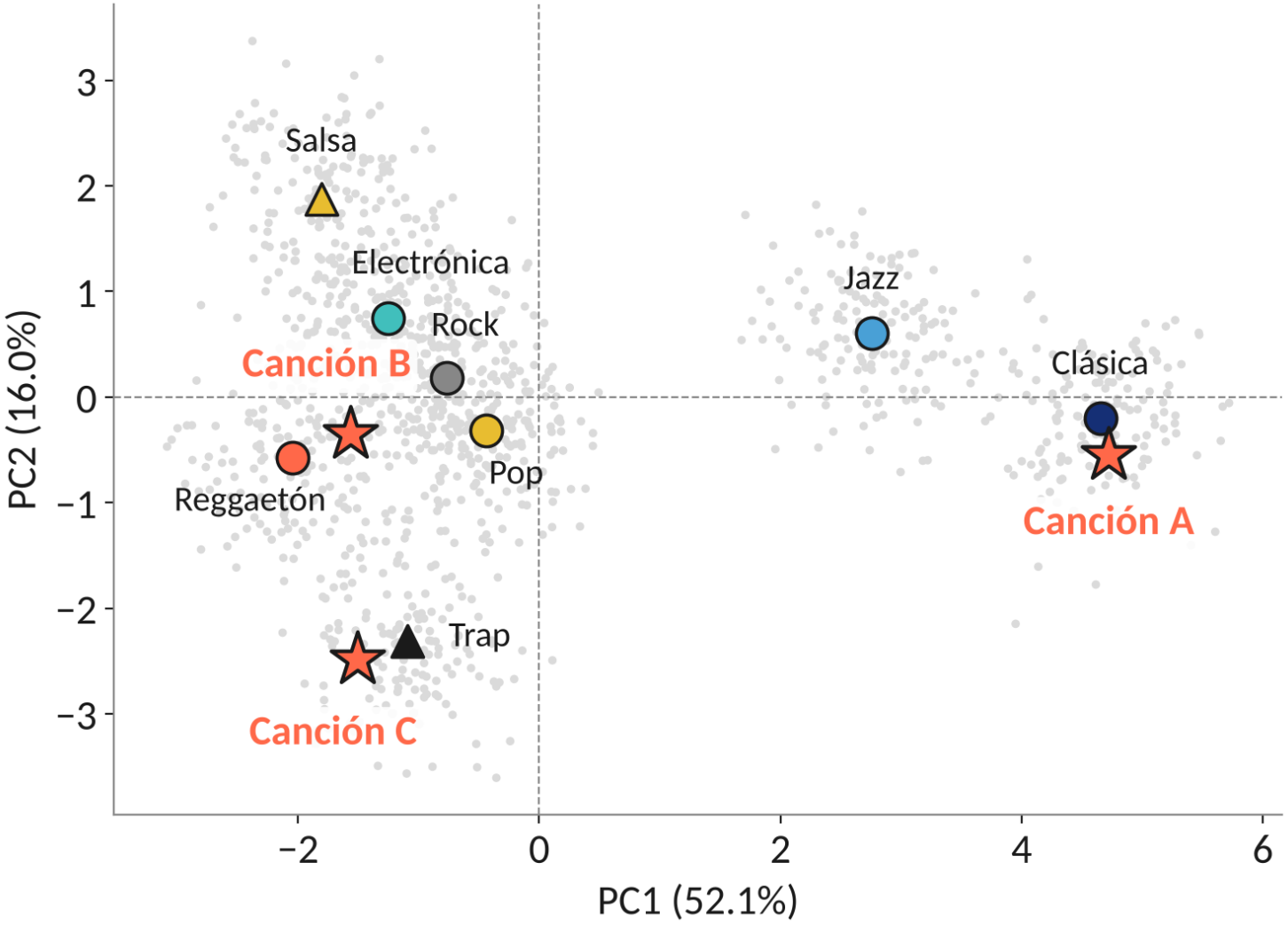
modelo[:-1] = todos los pasos menos el último (escalar + PCA)

```
nuevas = pd.DataFrame({
    'danceability': [0.30, 0.88, 0.72],
    'energy': [0.25, 0.82, 0.62],
    'loudness_db': [-22.0, -8.5, -10.0],
    'speechiness': [0.03, 0.12, 0.33],
    'acousticness': [0.85, 0.08, 0.12],
    'instrumentalness': [0.80, 0.01, 0.02],
    'liveness': [0.12, 0.15, 0.11],
    'valence': [0.25, 0.80, 0.40],
    'tempo_bpm': [95, 98, 140],
    'duration_ms': [215000, 180000, 170000]})
print(modelo[:-1].transform(nuevas))   # PC1, PC2
print(modelo.predict(nuevas))          # género cercano
```

A → Clásica (4.72, −0.55) · B → Reggaetón (−1.56, −0.35) · C → Trap (−1.51, −2.49)

Canciones Nuevas en el Mapa

Estrellas = canciones A, B y C proyectadas con el modelo exportado



Lectura

- A: acústica e instrumental → junto a Clásica.
- B: bailable y enérgica → junto a Reggaetón.
- C: mucha voz hablada (0.33) → junto a Trap.
- Se usan las medias y direcciones aprendidas con las 1 200 canciones.